Dynamic Multi-Criteria Request Routing and Load Balancing Architecture for 6G Network Slicing

1. Architectural Foundation and Operational Workflow

The transition toward sixth-generation (6G) cellular networks represents a fundamental evolution from network-centric connectivity to hyper-automated, service-oriented virtualized environments[1]. Central to this architecture is end-to-end (E2E) network slicing, which partitions a common physical substrate consisting of radio access networks (RAN), transport backhaul, and multi-access edge computing (MEC) nodes into dedicated logical networks[1]. Each slice operates as an autonomous virtual network tailored to specific Service Level Agreements (SLAs)[3].

The incoming service requests are categorized into three standardized baseline service types defined by the International Telecommunication Union (ITU-R) and 3GPP: Enhanced Mobile Broadband (eMBB), Ultra-Reliable Low-Latency Communications (URLLC), and Massive Machine-Type Communications (mMTC)[6]. To support heterogeneous applications—ranging from holographic telepresence to autonomous driving and industrial Internet of Things (IoT)—the load balancer cannot rely on traditional single-metric heuristics such as Round-Robin or Least-Connections[8]. Instead, it must execute slice-aware, multi-criteria request routing[11].

1.1 End-to-End System Architecture

The architectural framework consists of five distinct functional layers that operate synchronously to route traffic based on real-time node state telemetries and slice specifications:

- **Terminal and Device Layer**: Represents heterogeneous end-user equipment (UE), autonomous vehicles, industrial actuators, and massive IoT sensor arrays initiating service sessions across diverse spatial locations[2].
- **Slice Classification Gateway**: Ingests incoming packets and inspects the Single Network Slice Selection Assistance Information (S-NSSAI) header[5]. The S-NSSAI contains the Slice/Service Type (SST) indicator, which maps requests to eMBB (SST-1), URLLC (SST-2), or mMTC (SST-3) profiles[5].
- **6G Multi-Criteria Load Balancer Engine**: The core decision-making entity. It continuously ingests high-frequency telemetry metrics from available candidate processing nodes, normalizes incoming indicators, applies slice-specific priority weights, and evaluates candidate server suitability[9].
- **Edge Computing Layer (Candidate Nodes S_1, S_2, S_3, S_4)**: A distributed set of Multi-access Edge Computing (MEC) nodes situated at cell sites, regional aggregation points, or central offices[11]. Each node possesses distinct computational, memory,

caching, and database storage capacities[11].

- **Central Cloud and Content Delivery Network (CDN) Layer**: Connects edge nodes via backhaul transport networks to core data centers, accommodating overflow traffic, heavy cloud rendering, and global content repositories[14].

1.2 End-to-End Operational Execution Flow

The operational execution of a service request follows a structured multi-stage sequence:
First, a terminal device requests a session by transmitting data packets tagged with its assigned S-NSSAI identifier[5]. The Slice Classification Gateway intercepts the packet stream, extracts the SST parameter, and forwards the classified request to the 6G Multi-Criteria Load Balancer.
Second, the Load Balancer queries its local Telemetry Ingestion Cache, which maintains real-time metrics pulled from candidate edge nodes (S_1 through S_4) and transport network monitoring agents[1]. These metrics include compute utilization, queuing state, bandwidth availability, packet loss, and link latencies[9].
Third, the Load Balancer executes a mathematical normalization procedure, transforming raw multi-dimensional metrics into a standardized scalar domain between zero and one. Cost parameters (where lower values are preferable, such as latency or packet loss) are converted into standardized benefit scores[9].
Fourth, the Load Balancer selects a pre-configured or dynamically updated Analytic Hierarchy Process (AHP) weight vector corresponding to the request's slice type[9]. By multiplying the normalized metric matrix by the slice weight vector, the engine calculates a composite routing score for each candidate server[10].
Fifth, the Load Balancer selects the candidate server exhibiting the maximum composite score[11]. If the selected edge node has the required content cached or compute capacity available, it processes the task locally and returns the output directly to the device[14]. If an edge cache miss occurs or computational limits are breached, the edge node orchestrates a fast backhaul fetch from the Central Cloud/CDN layer before completing the service loop[14].

2. Comprehensive Parameter Investigation and Mathematical Normalization

To execute precise request routing, the load balancer evaluates fifteen distinct parameters across server-level compute capabilities, storage/database responsiveness, and transport network characteristics[9].

2.1 CPU Utilization (U_{cpu})

CPU utilization represents the percentage of central processing unit capacity currently consumed by active tasks on candidate server i, expressed as $U_{cpu,i} \in [0, 100\%]$. It reflects internal computational friction; high CPU load increases thread contention, context-switching latency, and task execution variance[9].
For eMBB slices, moderate-to-high CPU load causes minor performance degradation in video transcoding pipelines, but can be offset by large buffer spaces. For URLLC slices, elevated CPU

usage poses a catastrophic risk, as processing jitter can easily breach the stringent sub-millisecond end-to-end deadline[7]. For mMTC slices, CPU overhead per individual connection is minimal, allowing servers to process light IoT telemetry even under high overall processor utilization.

CPU utilization is a cost metric that must be minimized. It is normalized onto a $[0, 1]$ benefit scale using min-max inversion:

$$\hat{U}_{cpu,i} = \frac{U_{cpu,max} - U_{cpu,i}}{U_{cpu,max} - U_{cpu,min}}$$

where $U_{cpu,max} = 100\%$ and $U_{cpu,min} = 0\%$.
In the scoring function, CPU utilization carries high weight for URLLC services to ensure execution deterministic performance, moderate weight for eMBB compute tasks, and low weight for lightweight mMTC updates.

2.2 Memory Utilization (U_{mem})

Memory utilization measures the proportion of volatile RAM allocated to operational runtimes relative to total installed capacity on server i, denoted as $U_{mem,i} \in [0, 100\%]$. Exhausting memory forces operating systems to initiate page swapping to disk storage, causing severe performance degradation[9].
eMBB slices require substantial RAM to allocate buffer spaces for high-throughput stream processing and local media caching[4]. URLLC services demand reserved, unswapped memory allocations to prevent page-fault interrupts during critical execution steps[16]. mMTC applications require memory primarily to maintain concurrent session state records for massive numbers of connected devices[7].
Memory utilization is a cost metric that must be minimized. The parameter is normalized using linear min-max inversion:

$$\hat{U}_{mem,i} = \frac{U_{mem,max} - U_{mem,i}}{U_{mem,max} - U_{mem,min}}$$

In the candidate selection score, memory availability is prioritized for memory-intensive eMBB workloads and dense mMTC connection tables, preventing memory exhaustion failure modes.

2.3 Composite Server Load (L_{srv})

Composite server load is a multi-resource aggregate metric that quantifies total hardware stress on server i by integrating CPU usage, RAM allocation, disk I/O saturation, and internal bus traffic into a single scalar value $L_{srv,i} \in [0, 1]$. Single-resource metrics often fail to

capture system-wide bottlenecks; composite load offers a holistic view of node saturation[9]. All three slice types benefit from routing away from heavily loaded servers. However, URLLC traffic is especially sensitive to composite load due to resource preemption requirements[16], whereas eMBB and mMTC slices can tolerate higher operational loads if their primary requirements (bandwidth and connection slots, respectively) are met[7].
Composite server load is a cost metric that must be minimized. Because hardware performance degrades non-linearly near saturation, it is normalized using a logistic sigmoid decay function:

$$\hat{L}_{srv,i} = \frac{1}{1 + e^{\alpha(L_{srv,i} - L_{threshold})}}$$

where $L_{threshold}$ denotes the non-linear degradation knee (e.g., 0.75) and α represents the function steepness.
Composite load serves as a baseline filtering parameter across all slice scoring profiles to prevent node crash cascades.

2.4 Queue Length (Q_{len})

Queue length specifies the count of pending requests or network packets awaiting processing in server i's ingress buffers ($Q_{len,i}$). Queuing delay is directly proportional to queue length divided by server service rate ($D_q = Q_{len}/\mu$).
eMBB traffic can tolerate moderate queue depths due to client-side de-jitter buffers. URLLC traffic requires a near-zero queue length; arriving URLLC packets must access processing resources immediately or preempt existing non-critical tasks to avoid violating strict latency bounds[7]. mMTC traffic can accommodate deep queues, as non-real-time sensor updates are delay-tolerant[7].
Queue length is a cost metric that must be minimized. Due to the rapid accumulation of delay in deep queues, it is normalized using an exponential decay transformation:

$$\hat{Q}_{len,i} = \exp\left(-\lambda_q \cdot \frac{Q_{len,i}}{Q_{max}}\right)$$

where Q_{max} is the buffer ceiling and λ_q controls the penalty slope.
Queue length is assigned a high weight in URLLC scoring functions, rendering servers with long queues ineligible for latency-critical traffic.

2.5 Number of Active Requests (N_{act})

The number of active requests reflects the total count of concurrent open connection sockets,

microservice threads, or active execution contexts handled by server i ($N_{act,i}$). It measures thread concurrency limitations and socket descriptor availability[10].
For eMBB services, active request counts are typically moderate, though individual sessions consume significant data rates[7]. For URLLC services, high active request counts increase scheduling friction within the OS kernel. For mMTC services, active request capacity is the primary performance bottleneck, as 6G networks target connection densities of up to ten million devices per square kilometer[2].
Active requests is a cost metric that must be minimized relative to node capacity. It is normalized by scaling against maximum connection capacity N_{cap} :

$$\hat{N}_{act,i} = \frac{N_{cap} - N_{act,i}}{N_{cap}}$$

This parameter receives its highest weighting in mMTC selection profiles, steering incoming IoT connections to nodes with ample connection context space.

2.6 Database and Storage Latency (D_{db})

Database latency measures the average round-trip time required for server i to complete read and write transactions against attached local or distributed storage systems, expressed in milliseconds ($D_{db,i}$). It reflects storage subsystem performance and data retrieval responsiveness.
eMBB applications depend on low database latency for fast metadata lookups and catalog rendering during video streaming. URLLC applications rely on ultra-fast storage access for stateful processing, such as updating spatial models in autonomous driving networks[8]. mMTC applications write sensor readings asynchronously in batches, making them insensitive to database latency[7].
Database latency is a cost metric that must be minimized. It is normalized using linear min-max scaling:

$$\hat{D}_{db,i} = \frac{D_{db,max} - D_{db,i}}{D_{db,max} - D_{db,min}}$$

Database latency carries significant weight for stateful eMBB and URLLC transactions, but minimal weight for stateless or asynchronous mMTC updates.

2.7 Caching Availability (C_{cache})

Caching availability represents the unallocated high-speed storage volume (in Gigabytes) or

the presence of target content items pre-positioned on server i's edge cache ($C_{cache,i}$). Content caching near the network edge avoids unnecessary backhaul traversal[14].
For eMBB services, caching availability is a critical performance driver; serving high-definition video directly from local edge memory minimizes backhaul stress and delivery delay[4]. URLLC and mMTC services rarely rely on static content caches, as their payloads consist primarily of real-time control commands or dynamic telemetry data[7].
Caching availability is a benefit metric that must be maximized. It is normalized linearly relative to the maximum cache capacity among candidate nodes:

$$\hat{C}_{cache,i} = \frac{C_{cache,i}}{C_{cache,max}}$$

Caching availability is heavily weighted in eMBB scoring functions, steering media retrieval requests toward nodes with relevant cached assets.

2.8 Computation Availability (C_{comp})

Computation availability measures the unallocated floating-point and vector processing throughput available on server i ($C_{comp,i}$), measured in TeraFLOPS. It quantifies the node's capacity to execute compute-heavy workloads such as AI inference, graphics rendering, and signal processing[2].
URLLC applications rely on high compute availability to process complex algorithms (such as computer vision for autonomous navigation) within tight deadlines[7]. eMBB applications require compute availability for real-time video stitching and spatial rendering in Extended Reality (XR) scenarios[2]. mMTC applications require negligible compute capacity[7].
Computation availability is a benefit metric that must be maximized. It is normalized using linear scaling against peak available compute performance:

$$\hat{C}_{comp,i} = \frac{C_{comp,i}}{C_{comp,max}}$$

This parameter receives high weighting in URLLC and advanced eMBB routing matrices, directing compute-intensive tasks to nodes with available hardware acceleration.

2.9 Network Bandwidth and Available Throughput (B_{avail})

Available network bandwidth measures the unallocated transmission capacity along the network path to candidate server i, measured in Megabits or Gigabits per second ($B_{avail,i}$). It defines the data transfer rate achievable without inducing transport link congestion[8].
Bandwidth is the defining metric for eMBB slices, which demand peak data rates up to 1 Tbps and sustained user rates from 20 to 100 Gbps[2]. URLLC services require sufficient bandwidth to

prevent packet queuing, though their payload sizes are typically small[7]. mMTC services transmit small payloads (bytes to kilobytes) and require minimal bandwidth[7].
Available bandwidth is a benefit metric that must be maximized. It is normalized using linear min-max scaling:

$$\hat{B}_{avail,i} = \frac{B_{avail,i} - B_{min}}{B_{max} - B_{min}}$$

Bandwidth is the highest-weighted parameter in eMBB routing decisions, steering high-throughput streams to high-capacity network paths.

2.10 Network Latency / Round-Trip Time (L_{net})

Network latency quantifies the end-to-end transit delay incurred as packets travel between the user equipment and server i, measured in milliseconds ($L_{net,i}$). It includes propagation, serialization, and switching delays across transport links[12].
Network latency is the critical baseline KPI for URLLC slices, which require 6G latency bounds between 0.1 ms and 1 ms[2]. eMBB applications function adequately within a 10 ms to 50 ms latency envelope[4]. mMTC applications are delay-tolerant, operating effectively across latency ranges from hundreds of milliseconds to multiple seconds[7].
Network latency is a cost metric that must be minimized. Because URLLC SLAs enforce strict latency thresholds, it is normalized using a sigmoid function centered on target slice latency L_{target}:

$$\hat{L}_{net,i} = \frac{1}{1 + e^{\beta(L_{net,i} - L_{target})}}$$

where β dictates penalty severity for threshold breaches. Network latency carries dominant weight in URLLC selection profiles.

2.11 Physical Distance and Propagation Delay (D_{prop})

Physical distance measures the geographic separation between the client device and candidate server i, which directly dictates physical propagation delay ($D_{prop,i} \approx 5\ \mu\text{s per km}$ in optical fiber). Physical distance establishes an unyielding lower bound on network latency[14].
URLLC services require physical proximity, compelling the load balancer to choose local MEC nodes over distant regional data centers[19]. eMBB services can traverse moderate physical distances to reach centralized servers offering high caching capacity[4]. mMTC traffic can be routed to distant cloud facilities without degrading service quality[7].

Propagation delay is a cost metric that must be minimized. It is normalized using linear min-max scaling:

$$\hat{D}_{prop,i} = \frac{D_{prop,max} - D_{prop,i}}{D_{prop,max} - D_{prop,min}}$$

Physical distance serves as an initial spatial filter for URLLC routing decisions, restricting candidate nodes to those within an allowable geographic radius.

2.12 End-to-End Response Time (T_{resp})

End-to-end response time represents the complete operational duration from initial request transmission to final response delivery ($T_{resp,i}$). It aggregates all underlying delay components:

$$T_{resp,i} = L_{net,i} + D_{q,i} + T_{proc,i} + D_{db,i}$$

Response time reflects the actual performance experienced by the client[10].
URLLC applications rely on end-to-end response time as a primary metric to verify SLA compliance[3]. eMBB applications use it to track session responsiveness, while mMTC applications monitor it primarily to avoid connection timeout errors.
Response time is a cost metric that must be minimized. It is normalized using min-max scaling against the allowable SLA threshold T_{sla}:

$$\hat{T}_{resp,i} = \frac{T_{sla} - T_{resp,i}}{T_{sla} - T_{min}}$$

Response time carries high weight in URLLC scoring functions, prioritizing nodes that deliver low total end-to-end latency.

2.13 Packet Loss Rate (P_{loss})

Packet loss rate measures the percentage of transmitted packets that fail to reach their destination due to link noise, buffer overflows, or routing drop policies ($P_{loss,i} \in [0, 1]$).
Packet loss triggers retransmissions or error recovery mechanisms, increasing overall latency[18].
URLLC applications require extremely low packet loss to achieve 3GPP-specified reliability targets of 99.999% to 99.9999% [4]. eMBB applications tolerate minor packet loss through error correction and adaptive bitrates, though loss degrades peak throughput. mMTC applications easily tolerate packet loss, as missing sensor updates are updated in subsequent transmission cycles[7].

Packet loss rate is a cost metric that must be minimized. Because loss severely impacts URLLC reliability, it is normalized using an exponential decay function:

$$\hat{P}_{loss,i} = \exp(-\gamma \cdot P_{loss,i})$$

where γ is an exponential penalty factor. Packet loss is heavily weighted in URLLC selection functions to steer traffic toward stable transport paths.

2.14 Service Reliability and Availability (R_{avail})

Service availability measures operational uptime and successful request execution history on server i, expressed as a probability $R_{avail,i} \in [0.99, 0.999999]$. It quantifies hardware stability and operational fault tolerance[9].

URLLC applications demand ultra-high availability (99.999% to 99.9999%) to support mission-critical use cases such as remote surgery and smart grid protection[4]. eMBB applications operate adequately under standard commercial availability (99.9%). mMTC applications require high aggregate network availability, but individual node failures cause minimal impact due to device redundancy[7].

Service reliability is a benefit metric that must be maximized. Because target values approach unity (1.0), it is normalized using a logarithmic scale to emphasize small differences in high availability levels:

$$\hat{R}_{avail,i} = \frac{\log_{10}(1 - R_{avail,min}) - \log_{10}(1 - R_{avail,i})}{\log_{10}(1 - R_{avail,min}) - \log_{10}(1 - R_{avail,max})}$$

Reliability is heavily weighted in URLLC routing models to exclude nodes prone to transient failures.

2.15 Latency Variance / Jitter (σ^2_{lat})

Latency variance measures the statistical fluctuation of packet arrival times over a given transport path ($\sigma^2_{lat,i}$). High jitter destabilizes real-time control systems and forces receivers to expand buffer allocations.

URLLC applications require near-zero jitter to support deterministic control loops in tactile internet and industrial automation scenarios[7]. eMBB video streaming handles jitter using client-side playback buffers[4]. mMTC applications are insensitive to jitter[7].

Latency jitter is a cost metric that must be minimized. It is normalized using linear min-max scaling:

$$\hat{\sigma}^2_{lat,i} = \frac{\sigma^2_{max} - \sigma^2_{lat,i}}{\sigma^2_{max} - \sigma^2_{min}}$$

Jitter carries dedicated weight in URLLC scoring models, favoring transport paths with stable delay profiles.

2.16 Node Energy Efficiency (E_{node})

Node energy efficiency quantifies the computational work completed per unit of electrical energy consumed on server i ($E_{node,i}$), measured in operations per Joule or Watts per Gbps. It reflects overall green computing efficiency[8].

In 6G networks, energy efficiency is a central operational KPI, particularly for dense mMTC deployments where millions of small payloads must be processed with minimal power consumption[7]. For eMBB and URLLC services, energy efficiency is secondary to data throughput and latency guarantees[7].

Energy efficiency is a benefit metric that must be maximized. It is normalized using linear min-max scaling:

$$\hat{E}_{node,i} = \frac{E_{node,i} - E_{min}}{E_{max} - E_{min}}$$

Energy efficiency receives high weighting in mMTC routing decisions, helping optimize overall network operational costs.

3. Slice-Specific Quality of Service (QoS) and KPI Requirements

The core objective of dynamic request routing is translating standardized slice specifications into specific parameter priority profiles[1].

eMBB slices prioritize network bandwidth, caching capacity, memory buffer availability, and high compute performance[4]. These parameters ensure the network maintains sustained high-bitrate data transfers for immersive media, spatial computing, and high-definition video streams[2].

URLLC slices prioritize low network latency, minimal queue lengths, high reliability, zero latency jitter, low packet loss rates, and short physical propagation distances[7]. These metrics ensure the load balancer selects nodes capable of fulfilling strict, deterministic timing and fault-tolerance constraints[7].

mMTC slices prioritize active request capacity, composite server load, connection density, and node energy efficiency[7]. Throughput and latency metrics are deemphasized, enabling the load balancer to distribute lightweight IoT updates across energy-efficient edge nodes[7].

Key Performance Indicator / Metric	eMBB Slice Profile Target	URLLC Slice Profile Target	mMTC Slice Profile Target	Primary Routing Parameter
Peak Data Throughput Rate	20 Gbps – [cite: 2, 8]	1 – [cite: 7]	1 – [cite: 7]	Available Bandwidth (B_{avail})[18]
User Experienced Data Rate	100 Mbps – [cite: 2, 7]	10 Mbps [cite: 7]	10 kbps [cite: 7]	Available Bandwidth (B_{avail})[4]
End-to-End Latency Target	10 – [cite: 4]	0.1 – [cite: 2, 7, 8]	1 – (Delay Tolerant)[7]	Network Latency (L_{net})[19]
Service Reliability Target	99.9% (Standard)	99.999% – [cite: 4, 7, 20]	95.0% –	Reliability / Availability (R_{avail})[7]
Max Connection Density	10^4 devices/k	10^3 devices/k	10^7 devices/k [cite: 2, 4, 8]	Active Requests (N_{act})[7]
Allowed Latency Jitter	$\leq$	$\leq$ (μs level)	Unconstrained	Latency Jitter (σ^2_{lat})[19]
Primary System Resource	Cache (C_{cache}) & RAM (U_{mem})[4]	Compute (C_{comp}) & CPU (U_{cpu})[8]	Energy (E_{node}) & Sockets (N_{act})[8]	Hardware Capacity Matching

4. Multi-Criteria Decision-Making (MCDM) Scoring Framework

To consolidate normalized metrics into a single selection score (S_i) for candidate node i, the load balancer implements a Multi-Criteria Decision-Making (MCDM) algorithm[9].

4.1 Weighted Sum Model (WSM) Formulation

The dynamic score $S_i^{(SST)}$ for candidate server i under slice type $SST \in \{\text{eMBB}, \text{URLLC}, \text{mMTC}\}$ is calculated as:

$$S_i^{(SST)} = \sum_{k=1}^{K} w_k^{(SST)} \cdot \hat{p}_{k,i}$$

subject to the weight normalization constraint:

$$\sum_{k=1}^{K} w_k^{(SST)} = 1.0, \quad \text{and} \quad w_k^{(SST)} \geq 0, \quad \forall k$$

where:

- K represents the total number of evaluated metrics ($K = 16$).
- $\hat{p}_{k,i} \in [0, 1]$ represents the normalized, direction-adjusted value of metric k for server i.
- $w_k^{(SST)}$ denotes the dynamic weight assigned to metric k for slice type SST, calculated using the Analytic Hierarchy Process (AHP) or Entropy Weight Method[9].

4.2 Dynamic Weight Vectors ($W^{(SST)}$) Across Slices

The table below outlines the normalized weight distributions assigned to each metric across slice types[9]:

Parameter Evaluated	Parameter Symbol	eMBB Slice Weight (wk(eMBB))	URLLC Slice Weight (wk(URLLC))	mMTC Slice Weight (wk(mMTC))
CPU Utilization	U_{cpu}	0.04	0.08	0.03
Memory	U_{mem}	0.06	0.04	0.03

Utilization				
Composite Load	L_{srv}	0.05	0.05	0.08
Queue Length	Q_{len}	0.02	**0.14**	0.01
Active Requests	N_{act}	0.03	0.02	**0.25**
Database Latency	D_{db}	0.05	0.04	0.01
Caching Availability	C_{cache}	**0.24**	0.01	0.01
Compute Availability	C_{comp}	0.10	0.08	0.01
Available Bandwidth	B_{avail}	**0.25**	0.04	0.01
Network Latency	L_{net}	0.05	**0.18**	0.01
Propagation Delay	D_{prop}	0.02	0.05	0.01
Response Time	T_{resp}	0.03	**0.10**	0.01
Packet Loss Rate	P_{loss}	0.02	**0.08**	0.01
Service Reliability	R_{avail}	0.02	**0.05**	0.02
Latency Jitter	σ^2_{lat}	0.01	**0.04**	0.00

Energy Efficiency	E_{node}	0.01	0.00	**0.50**
Aggregate Sum	$\sum w_k$	**1.00**	**1.00**	**1.00**

4.3 TOPSIS Compromise Ranking Integration

To avoid node starvation and manage trade-offs among competing parameters, the load balancer incorporates the Technique for Order of Preference by Similarity to Ideal Solution (TOPSIS)[9].

First, the engine constructs a weighted normalized decision matrix V, where each element v_{ij} is defined as:

$$v_{ij} = w_j^{(SST)} \cdot \frac{x_{ij}}{\sqrt{\sum_{i=1}^{M} x_{ij}^2}}$$

where x_{ij} represents the raw metric score of server i for metric j, and M is the number of candidate nodes[10].

Second, the engine determines the Positive Ideal Solution (V^+) and Negative Ideal Solution (V^-):

$$V^+ = \{v_1^+, v_2^+, \ldots, v_K^+\}, \quad \text{where } v_j^+ = \max_i(v_{ij}) \text{ for benefit metrics, and } \min_i(v_{ij}) \text{ for cost m}$$

$$V^- = \{v_1^-, v_2^-, \ldots, v_K^-\}, \quad \text{where } v_j^- = \min_i(v_{ij}) \text{ for benefit metrics, and } \max_i(v_{ij}) \text{ for cost m}$$

Third, the engine calculates the Euclidean distances from each candidate node to the ideal solutions:

$$D_i^+ = \sqrt{\sum_{j=1}^{K}(v_{ij} - v_j^+)^2}, \quad D_i^- = \sqrt{\sum_{j=1}^{K}(v_{ij} - v_j^-)^2}$$

Fourth, the relative closeness coefficient (C_i) is computed for each candidate server:

$$C_i = \frac{D_i^-}{D_i^+ + D_i^-}, \quad \text{where } C_i \in [0, 1]$$

The load balancer routes the request to the candidate node exhibiting the highest closeness coefficient C_i , ensuring the selection reflects the optimal compromise between performance constraints and current infrastructure conditions[9].

5. Conclusions

This study demonstrates that request routing and load balancing within 6G network slicing architectures require a shift from conventional load-centric scheduling to dynamic multi-criteria decision models[11]. By parameterizing node compute profiles, database responsiveness, and network transport indicators, the proposed framework aligns routing decisions directly with slice-specific SLAs[3].

Integrating mathematical normalization techniques with slice-weighted AHP-TOPSIS scoring mechanisms enables the load balancer to balance trade-offs across competing network constraints[9]. eMBB traffic is routed toward high-throughput, cache-enabled edge nodes; URLLC traffic is dispatched to low-latency, highly reliable processing environments; and mMTC traffic is distributed across high-capacity, energy-efficient infrastructure nodes[4].

This dynamic request-routing architecture improves overall resource utilization, prevents localized edge node congestion, and preserves end-to-end slice isolation across next-generation wireless networks[1].

Works cited

1. Customized Slicing for 6G: Enforcing Artificial Intelligence on ... - arXiv, https://arxiv.org/pdf/2102.10498
2. AI-Native Network Slicing for 6G Networks - arXiv, https://arxiv.org/pdf/2105.08576
3. Balancing resource utilization and slice dissatisfaction through, https://pmc.ncbi.nlm.nih.gov/articles/PMC12217554/
4. Optimizing 5G Network Slicing: An End-to-End Approach with, https://www.eurecom.fr/publication/7888/download/comsys-publi-7888.pdf
5. 5G Network Slicing. A Complete Technical Guide - Medium,

https://medium.com/@aayushbhatnagar_10462/5g-network-slicing-4447642f79e8
6. Towards 6G Network Slicing - arXiv, https://arxiv.org/html/2412.11366v1
7. eMBB vs URLLC vs mMTC: 5G Pillars - 5G/6G Academy, https://www.5g6gacademy.com/compare/embb-vs-urllc-vs-mmtc
8. GAN-Enhanced Deep Deterministic Policy Gradient Framework for, https://arxiv.org/pdf/2604.08576
9. (PDF) AHP-TOPSIS-Based Reliable Switch Migration Algorithm for, https://www.researchgate.net/publication/396917639_AHP-TOPSIS-Based_Reliable_Switch_Migration_Algorithm_for_Software-Defined_Networks
10. Adaptive Multi-Level Cloud Service Selection and Composition, https://www.mdpi.com/2076-3417/15/20/11010
11. A Multi-Criteria Decision Making Method for Network Slice Edge, https://www.researchgate.net/publication/342521785_A_Multi-Criteria_Decision_Making_Method_for_Network_Slice_Edge_Infrastructure_Selection
12. (PDF) Multi-Criteria Decision Making for Resource Allocation in 5G, https://www.researchgate.net/publication/404872053_Multi-Criteria_Decision_Making_for_Resource_Allocation_in_5G-Enabled_Edge_Computing_Using_Fuzzy_Logic_Approaches
13. A Novel Approach to Multi-Provider Network Slice Selector for 5G, https://pmc.ncbi.nlm.nih.gov/articles/PMC9412514/
14. Efficient Schemes for Optimizing Load Balancing and ... - MDPI, https://www.mdpi.com/2078-2489/15/11/670
15. Joint Network Slicing, Routing, and In-Network Computing for ... - arXiv, https://arxiv.org/pdf/2401.06306
16. Resource Slicing for eMBB and URLLC Services in Radio Access, https://people.ece.ubc.ca/vincentw/J/SBW-TWC-2022.pdf
17. (PDF) Joint Network Slicing, Routing, and In-Network Computing for, https://www.researchgate.net/publication/381888388_Joint_Network_Slicing_Routing_and_In-Network_Computing_for_Energy-Efficient_6G
18. Building 5G Networks - A Quick and Practical Guide to Network Slicing, https://www.vamsitalkstech.com/5g/building-5g-networks-a-quick-and-practical-guide-to-network-slicing/
19. 5G Ultra-Reliable Low-Latency Communication Implementation, https://www.mdpi.com/2079-9292/8/9/981
20. Exploring the key technologies and applications of 6G wireless, https://pmc.ncbi.nlm.nih.gov/articles/PMC12018058/